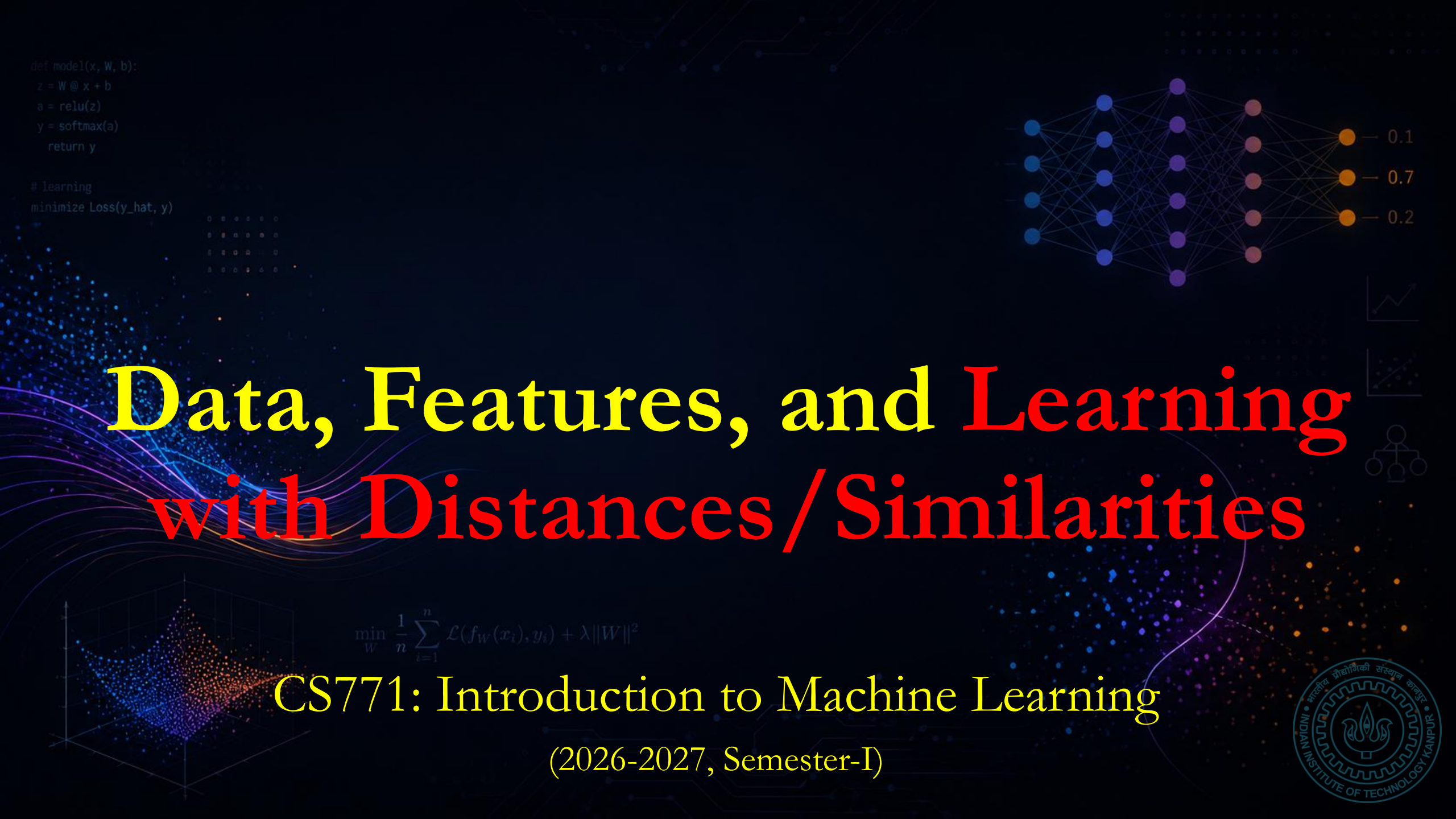


Data, Features, and Learning with Distances/Similarities

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

Plan Today

- Data and Features
  - Inputs
  - Outputs/labels/responses
- Basic notation and operations
  - Vectors and operations on them
  - Computing distances
  - Computing similarities
- A simple ML algorithm based only on distance/similarity computation
  - Learning with Prototypes (LwP)



Data, Features, and Labels/Responses

In ML, a dataset is a collection of "examples"

1 INPUT FOR EXAMPLE n



home n

features of x_n

- area
- bedrooms
- location

x_n = input for example n

$x_{n,d}$ = d-th feature of input n

2 IS THE EXAMPLE LABELED?

UNLABELED

x_n only

input available



LABELED

(x_n, y_n)

input + output



y_n = output / response

also called target or label

3 A DATASET = MANY EXAMPLES

n	area	bedrooms	location	price
1	120 m²	3	Kanpur	₹72 L
2	85 m²	2	Lucknow	₹48 L
⋮	⋮	⋮	⋮	⋮
N	150 m²	4	Varanasi	₹90 L

input x_n — D features

output y_n — if labeled

UNSUPERVISED LEARNING

$$\mathcal{D} = \{ x_n \}_{n=1}^{N}$$

inputs only

SUPERVISED LEARNING

$$\mathcal{D} = \{ (x_n, y_n) \}_{n=1}^{N}$$

input–output pairs

A dataset contains N examples; labels determine the learning setting.

Feature Vector

1 D Features of Example n



example n

feature 1: $x_{n,1}$

feature 2: $x_{n,2}$

feature 3: $x_{n,3}$

⋮

feature D: $x_{n,D}$

Each $x_{n,d}$ is one scalar feature value.

2 Choose a Fixed Order

area

bedrooms

house age

distance

⋮

The same feature must occupy the same position for every example.

3 Collect Them into a Vector

$$\mathbf{x}_n = \begin{bmatrix} x_{n,1} \\ x_{n,2} \\ \vdots \\ x_{n,D} \end{bmatrix}$$

$\mathbf{x}_n \in \mathbb{R}^D$

Size: $D \times 1$

D = number of features

$\mathbf{x}_n$: feature vector representing example n

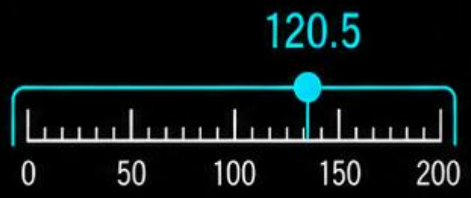
$x_{n,d}$: feature d of example n

D features $\Rightarrow$ D-dimensional vector

Unless stated otherwise, vectors are column vectors.

Feature Types: Some Common Examples

REAL-VALUED

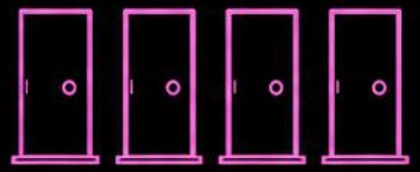


$x_{n,d} \in \mathbb{R}$

Continuous measurement

area $= 120.5\,\mathrm{m}^2$

COUNT



$x_{n,d} \in \{0, 1, 2, \ldots\}$

Non-negative integer

bedrooms $= 4$

BINARY



$x_{n,d} \in \{0, 1\}$

Yes / no

has garage $= 1$

CATEGORICAL



$x_{n,d} \in \{\text{Kanpur}, \text{Lucknow}, \text{Varanasi}\}$

Unordered categories

location $=$ Kanpur

ORDINAL



$x_{n,d} \in \{\text{poor} < \text{fair} < \text{good} < \text{excellent}\}$

Ordered categories

condition $=$ good

Feature type determines which operations and encodings make sense.

Scale numerical features when needed; encode categories without inventing false numerical meaning.

Output/Response/Label Types

For a labeled example, y_n need not be a single real number

REAL-VALUED

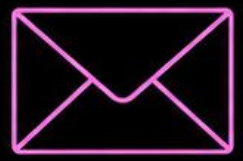


$y_n \in \mathbb{R}$

Scalar

house-price prediction

BINARY



spam | not spam

$y_n \in \{0, 1\}$

Two classes

spam detection

CATEGORICAL



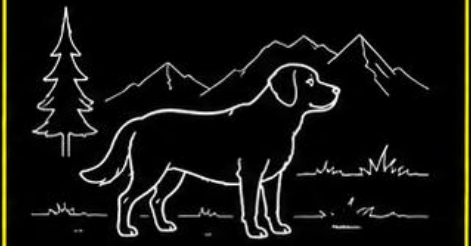
0 | 1 | 2 | ⋯ | 9

$y_n \in \{1, \ldots, K\}$

One of K classes

digit recognition

MULTI-LABEL



☑ dog | ☑ outdoor | ☐ car

$y_n \in \{0, 1\}^K$

Any subset of K labels

image tagging

GRAPH-STRUCTURED



$y_n = (V_n, E_n)$

Nodes + edges

molecular generation

The form of y_n determines the prediction task—and often the loss and model output.

Outputs may be scalars, labels, sets of labels, or structured objects.

Inputs can have different shapes and structures

TABULAR INPUT



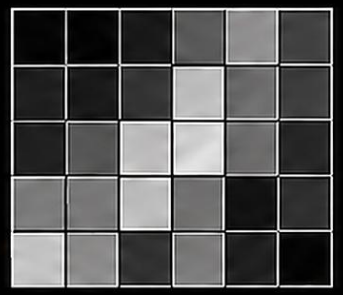
- area
- bedrooms
- house age
- distance

$\boldsymbol{x}_n \in \mathbb{R}^D$

Vector

D scalar features

GRAYSCALE IMAGE

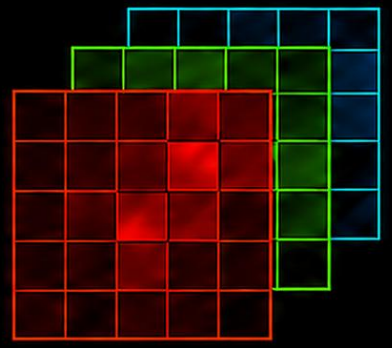


$\boldsymbol{x}_n \in \mathbb{R}^{H \times W}$

Matrix

height × width

COLOUR IMAGE

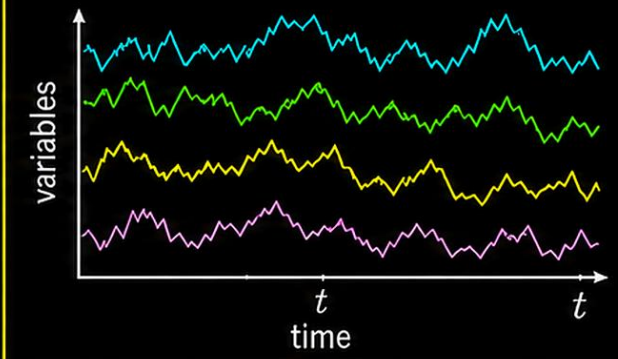


$\boldsymbol{x}_n \in \mathbb{R}^{C \times H \times W}$

3-D Tensor

channels × height × width

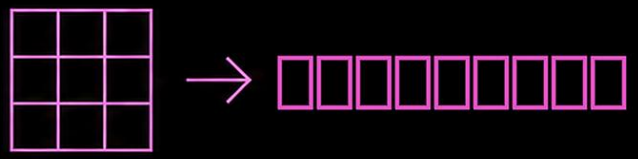
MULTIVARIATE TIME SERIES



$\boldsymbol{x}_n \in \mathbb{R}^{T \times D}$

Matrix

time × variables



FIXED-SIZE ARRAYS CAN BE FLATTENED

Flattening enables vector-based methods, but explicit spatial or temporal structure is no longer preserved.

Same notation $\boldsymbol{x}_n$; different shape and structure.

Use Raw Input Features or Extracted Features?



Extracted features capture semantics better and are usually more compact as well

RAW INPUT
pixels



1 Domain-Designed Feature Extraction



edges • textures • shape • colour

hand-crafted features

Designed using knowledge of the input and the task.

2 Learned Feature Extraction



deep feature extractor

learned features

Patterns are learned automatically from data.

ML MODEL

uses the extracted features

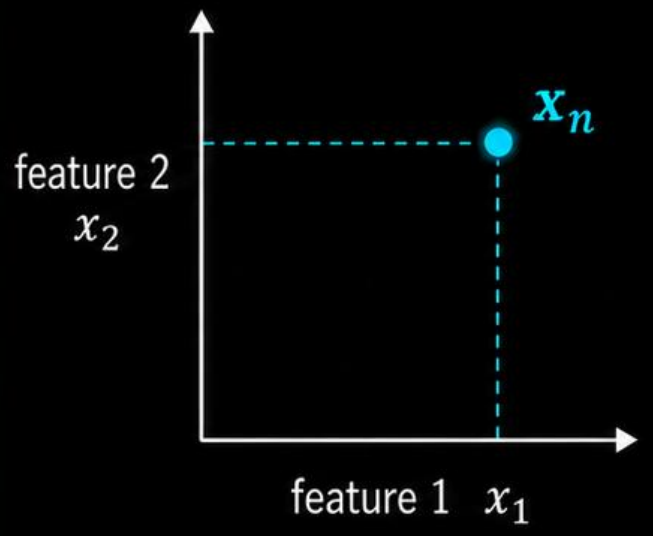
Useful, task-relevant features often must be extracted from the raw input.
Domain knowledge can design them; deep learning can discover them from data.
What is useful depends on the task.

Data Geometry

D feature values locate one example in a D-dimensional feature space.

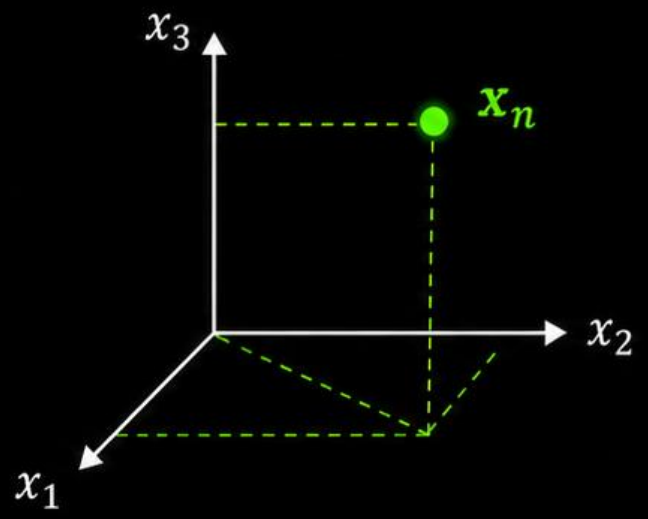
A. Two features → 2-D space

$$\mathbf{x}_n = [\, x_{n,1},\ x_{n,2} \,]^T$$



B. Three features → 3-D space

$$\mathbf{x}_n = [\, x_{n,1},\ x_{n,2},\ x_{n,3} \,]^T$$

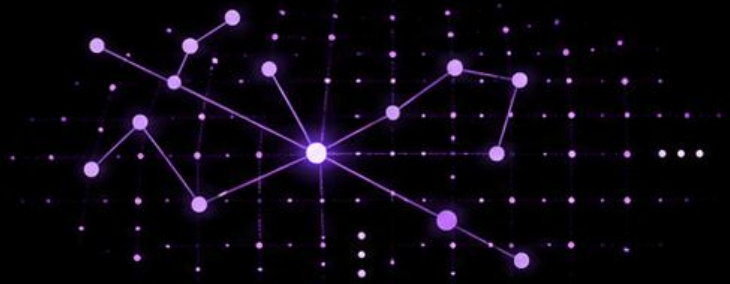


C. D features → D-Dimensional Space

$x_{n,1}$	$x_{n,2}$	$x_{n,3}$	$\cdots$	$x_{n,D}$

$$\mathbf{x}_n \in \mathbb{R}^D$$

Cannot be drawn directly,
but the same geometry applies.



D entries = D coordinates = dimension D

Each example becomes one point in feature space.

Vectors: Some Basic Operations

1 ADDITION AND SUBTRACTION

For $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^D$, addition and subtraction are componentwise.

ADDITION

$$\boldsymbol{a} + \boldsymbol{b} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \\ \vdots \\ a_D + b_D \end{bmatrix}$$

$$(\boldsymbol{a} + \boldsymbol{b})_i = a_i + b_i$$

SUBTRACTION

$$\boldsymbol{a} - \boldsymbol{b} = \begin{bmatrix} a_1 - b_1 \\ a_2 - b_2 \\ \vdots \\ a_D - b_D \end{bmatrix}$$

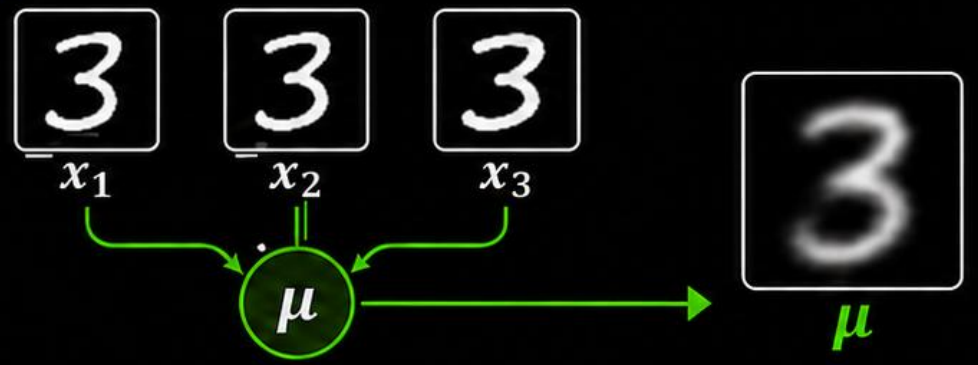
$$(\boldsymbol{a} - \boldsymbol{b})_i = a_i - b_i$$

$$\boldsymbol{a} \pm \boldsymbol{b} \in \mathbb{R}^D$$

The result has the same dimension D.

☆ Both vectors must have the same dimension.

2 MEAN (AVERAGE / CENTROID)



$$\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n$$

$\boldsymbol{\mu} \in \mathbb{R}^D$

The mean has the same dimension as every input vector.

☆ Mean = representative prototype / centroid

⚠ For images, a pixel-wise mean is meaningful only when examples are aligned.

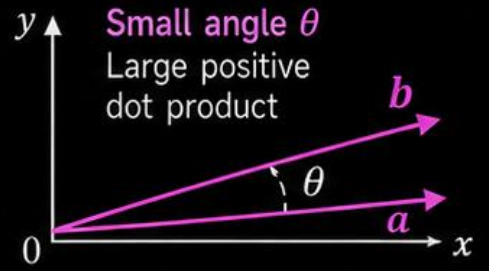
☆ Vector arithmetic **combines examples**; the mean **summarizes** a collection.

Vectors: Some Basic Operations

1 DOT (INNER) PRODUCT

$$\langle \boldsymbol{a}, \boldsymbol{b} \rangle = \boldsymbol{a}^T \boldsymbol{b} = \sum_{i=1}^{D} a_i b_i \in \mathbb{R}$$

The dot product returns a scalar.





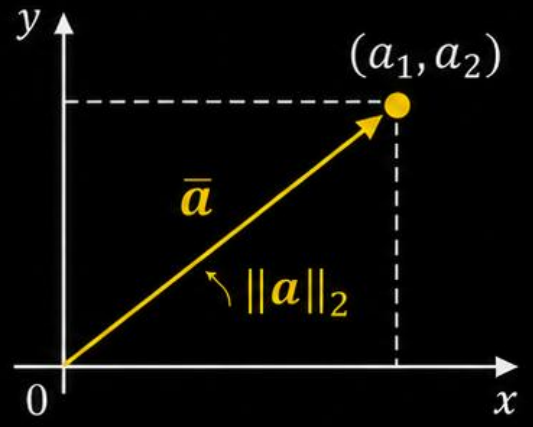
$$\boldsymbol{a}^T \boldsymbol{b} = \|\boldsymbol{a}\|_2 \, \|\boldsymbol{b}\|_2 \cos\theta$$



For unit-normalized vectors, dot product = cosine similarity.

2 EUCLIDEAN (L2) NORM

$$\|\boldsymbol{a}\|_2 = \sqrt{\langle \boldsymbol{a}, \boldsymbol{a} \rangle} = \sqrt{\boldsymbol{a}^T \boldsymbol{a}} = \sqrt{\sum_{i=1}^{D} a_i^2}$$



The norm is the vector's length—the Euclidean distance from the origin.

$$\|\boldsymbol{a}\|_2 \geq 0$$

$$\|\boldsymbol{a}\|_2 = 0 \text{ only when } \boldsymbol{a} = \boldsymbol{0}$$

Also called the **L2** norm.

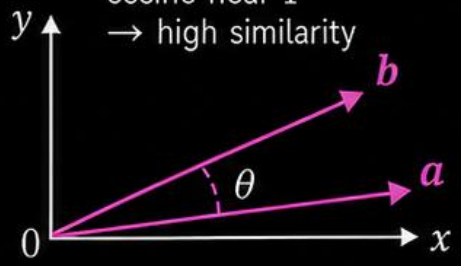
Dot product → alignment | **Norm** → length

Vectors: Computing Pairwise Similarities

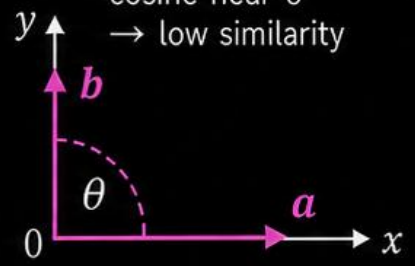
1 COSINE SIMILARITY

$$\cos(\boldsymbol{a}, \boldsymbol{b}) = \frac{\langle \boldsymbol{a}, \boldsymbol{b} \rangle}{\|\boldsymbol{a}\|_2 \, \|\boldsymbol{b}\|_2} = \frac{\boldsymbol{a}^{\mathrm{T}} \boldsymbol{b}}{\|\boldsymbol{a}\|_2 \, \|\boldsymbol{b}\|_2}$$

Small angle
cosine near 1
→ high similarity



Near 90°
cosine near 0
→ low similarity



Cosine compares **direction**, largely ignoring magnitude.

For nonzero vectors, $-1 \le \cos(\boldsymbol{a}, \boldsymbol{b}) \le 1$.

☆ For **unit-normalized** vectors: $\cos(\boldsymbol{a}, \boldsymbol{b}) = \langle \boldsymbol{a}, \boldsymbol{b} \rangle = \boldsymbol{a}^{\mathrm{T}} \boldsymbol{b}$



similar cat embeddings → **HIGH**

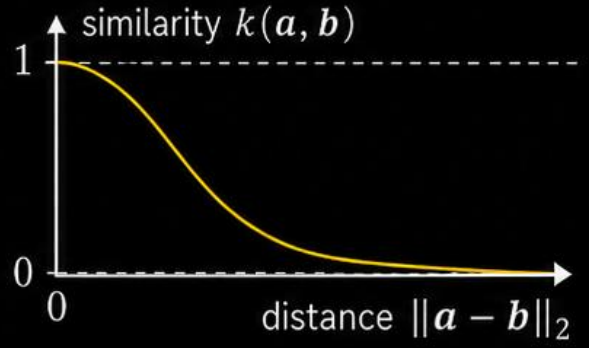


cat and car embeddings → **LOW**

2 RBF (GAUSSIAN) KERNEL

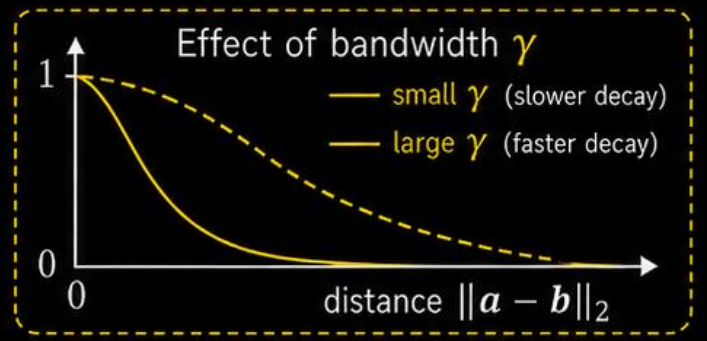
A nonlinear similarity built from Euclidean distance.

$$k(\boldsymbol{a}, \boldsymbol{b}) = \exp\left(-\gamma \, \|\boldsymbol{a} - \boldsymbol{b}\|_2^2\right), \quad \gamma > 0$$



close vectors → $k \approx 1$
far vectors → $k \approx 0$

Larger γ makes similarity decay faster.



More on kernels later.

☆ **Distance ↑ ⇒ Similarity ↓** | The chosen measure **defines** what **"close"** means.

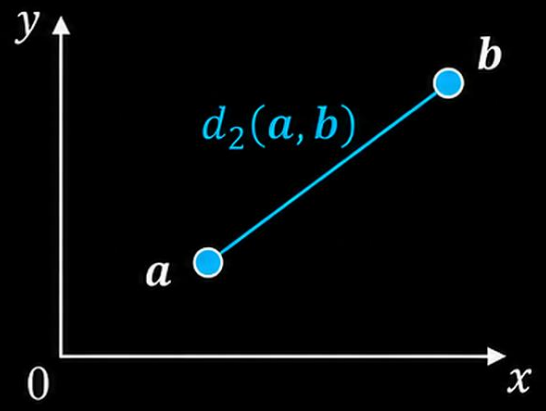
Vectors: Computing Pairwise Distances

1 EUCLIDEAN (L2) DISTANCE

For $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^D$, distance measures how far apart they are.

$$d_2(\boldsymbol{a}, \boldsymbol{b}) = \|\boldsymbol{a} - \boldsymbol{b}\|_2 = \sqrt{\sum_{i=1}^{D} (a_i - b_i)^2}$$

$$d_2^2(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} - \boldsymbol{b})^\top (\boldsymbol{a} - \boldsymbol{b})$$



small distance → nearby
large distance → far apart

$$d_2(\boldsymbol{a}, \boldsymbol{b}) \geq 0$$

$$d_2(\boldsymbol{a}, \boldsymbol{b}) = d_2(\boldsymbol{b}, \boldsymbol{a})$$

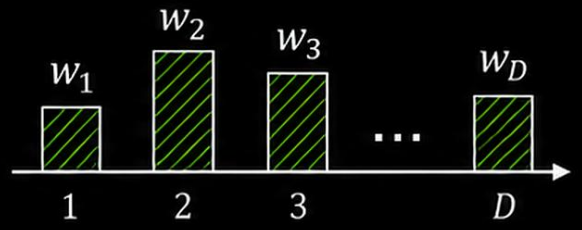
Result: one scalar

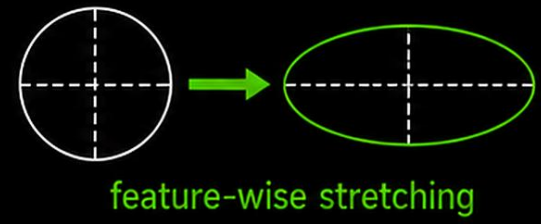
2 WEIGHTED EUCLIDEAN DISTANCE

Different features can contribute with different importance.

$$d_W(\boldsymbol{a}, \boldsymbol{b}) = \sqrt{\sum_{i=1}^{D} w_i (a_i - b_i)^2} = \sqrt{(\boldsymbol{a} - \boldsymbol{b})^\top \mathbf{W} (\boldsymbol{a} - \boldsymbol{b})}$$

$$\mathbf{W} = \mathrm{diag}(w_1, \ldots, w_D), \quad w_i > 0$$





feature-wise stretching

Large w_i → differences in feature i matter more.
Weights may be specified or learned from data.

⚠ Feature scale also changes distance—standardization is often essential.

☆ The chosen **distance defines** what **"nearby"** means. 💡

Handle Features Carefully in Distances/Similarities

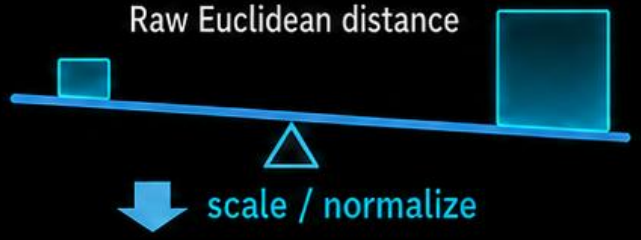
Useful features must respect **scale**, **structure**, and **relevance**.

1 Align the Scales

Age: 20–80 Salary: 20,000–2,000,000



Raw Euclidean distance

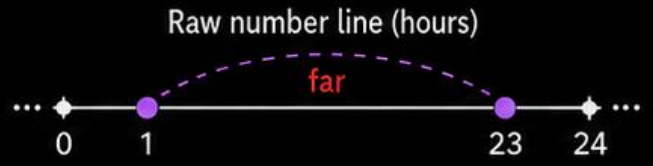




Large numerical ranges can dominate distance.

Compare features on meaningful scales.

2 Encode the Structure



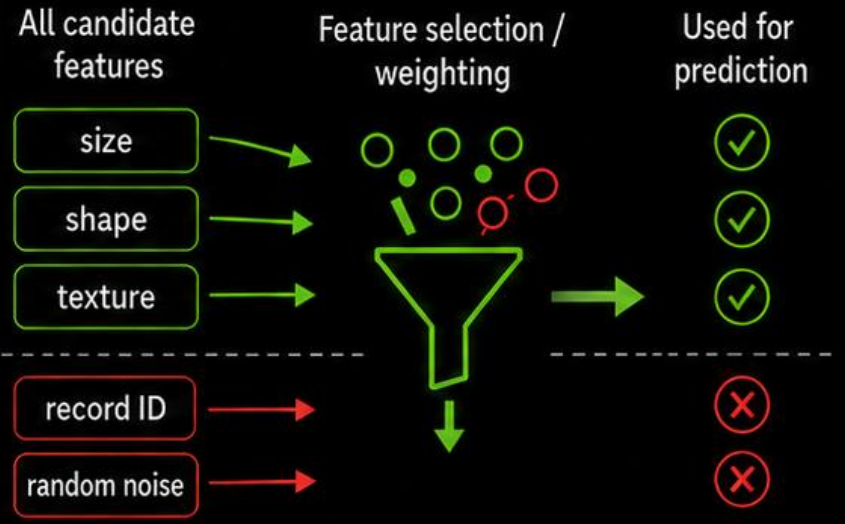


Hour 23 is close to hour 1.

$$h \rightarrow \left(\sin\left(\frac{2\pi h}{24}\right), \cos\left(\frac{2\pi h}{24}\right)\right)$$

Raw numbers may hide the true geometry.

3 Keep What Matters



Irrelevant features can distort neighborhoods.

Select or weight features according to the task.

For distance-based methods, every included feature contributes to the distance.

Encode structure, align scales, and remove irrelevant information before comparing examples.

Supervised Learning: Regression and Classification

Supervised learning uses input–target pairs (x_n, y_n). The type of target determines the task.

1 Regression

Target y is real-valued.

input $\mathbf{x}$



output y (real-valued)

Dog weight = 6.2 kg



$y \in \mathbb{R}$

Examples: weight, price, temperature

2 Binary Classification

Target chooses one of two classes.

input $\mathbf{x}$



Dog or cat?





$y \in \{0, 1\}$

Exactly one class is correct.

3 Multiclass Classification

Target chooses one of $K > 2$ classes.

input $\mathbf{x}$



Predict the dog breed







A common encoding: one-hot vector of length K

$y = [1, 0, 0, \ldots, 0]$

… Exactly one class is correct.

4 Multilabel Classification (also called tagging)

Several labels may be true simultaneously.

input $\mathbf{x}$



Dog Fluffy Sitting Collar Running

$y = [1, 1, 1, 0, 0]$

Binary label vector of length L

$L > 1$ labels, each independently binary

$L = 5$

Other supervised tasks also exist—for example, ranking—which we will study later.

Regression → **real-valued** target | **Classification** → discrete / categorical target

Same input; different target structure → different learning problem

Our First ML Algorithm

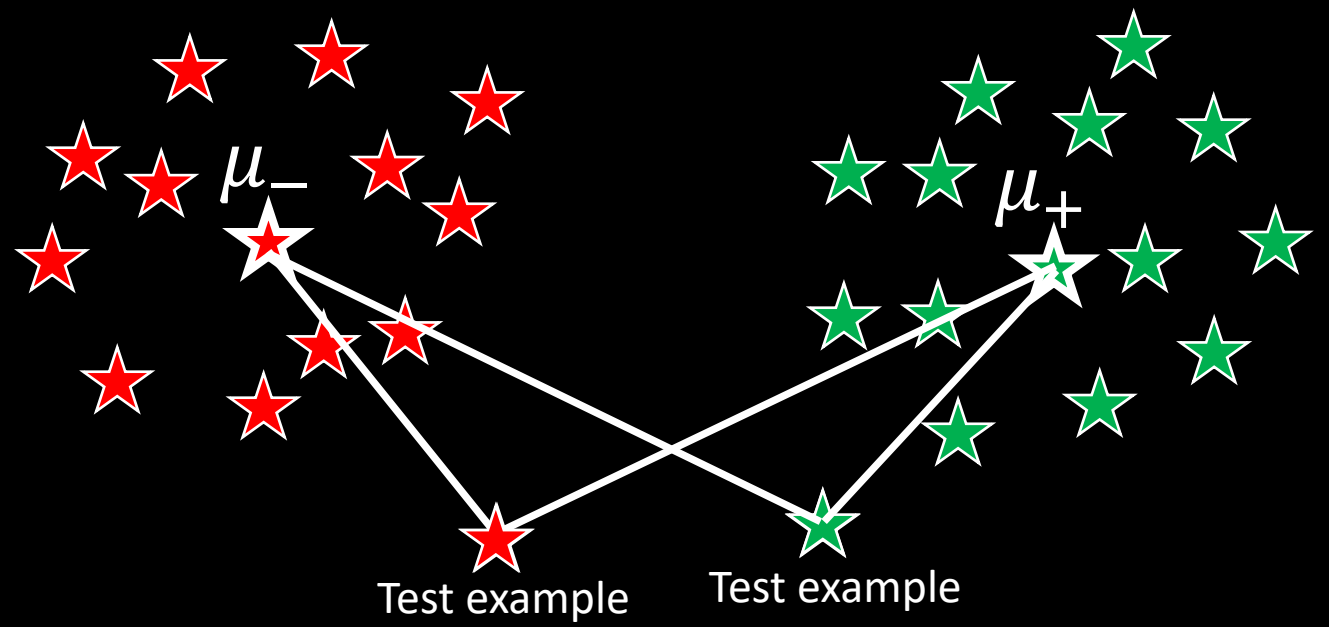


Learning with Prototypes (LwP)

A very simple algorithm for supervised learning (classification problems)

$$\mu_- = \frac{1}{N_-} \sum_{y_n=-1} \mathbf{x}_n$$

$$\mu_+ = \frac{1}{N_+} \sum_{y_n=+1} \mathbf{x}_n$$



Red examples have label -1

Green examples have label +1

LwP Example (Handwritten Digits Classification)

A simple classification algorithm: summarize each class by its mean feature vector.

CLASSIFICATION ONLY • TRAINING STEP

Example task: handwritten-digit classification | Training data: $\{(\mathbf{x}_n, y_n)\}_{n=1}^{N}, \quad \mathbf{x}_n \in \mathbb{R}^D, \quad y_n \in \{1, \ldots, K\}$



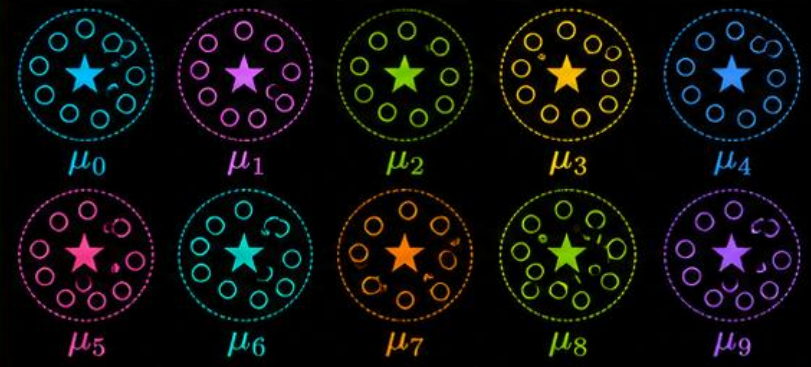
Illustration: pixel-wise means of aligned digit images.

Class Prototype

$$\mu_k = \frac{1}{N_k} \sum_{n\,:\,y_n = k} \mathbf{x}_n$$

N_k = number of training examples in class k

K classes → K prototype vectors



○ training example ★ prototype

The model stores only the class prototypes.

Also called nearest-centroid or nearest-class-mean classification.

Training compresses every class into one representative vector.

LwP Example (Handwritten Digits Classification)

For a new input, compare it with every class prototype and choose the closest one.

CLASSIFICATION ONLY • PREDICTION STEP

1 New Input



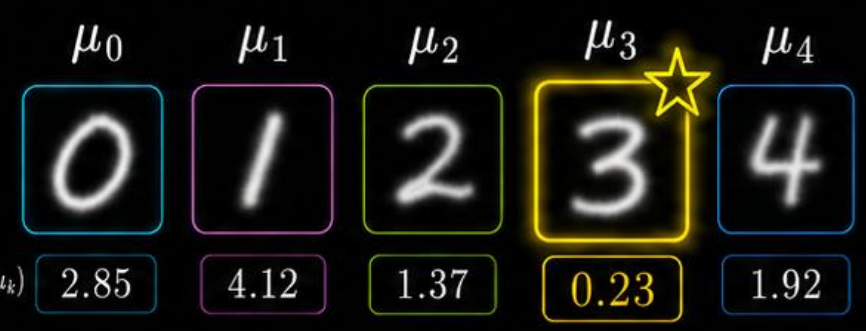
$\mathbf{x}$

unlabelled test input

feature vector

$\mathbf{x} \in \mathbb{R}^D$

2 Compare with Stored Prototypes



Compute one distance per class.

3 Predict

The closest prototype determines the label.

Winning prototype



$\hat{y} = 3$

predicted class

Prediction Rule

$$d_k = d(\mathbf{x}, \mu_k)$$

$$\hat{y} = \arg\min_{k \in \{1,\dots,K\}} d(\mathbf{x}, \mu_k)$$

With Euclidean distance:

$$d(\mathbf{x}, \mu_k) = \|\mathbf{x} - \mu_k\|_2$$

Smallest distance wins.

The prototypes are learned during training; prediction only compares and selects.

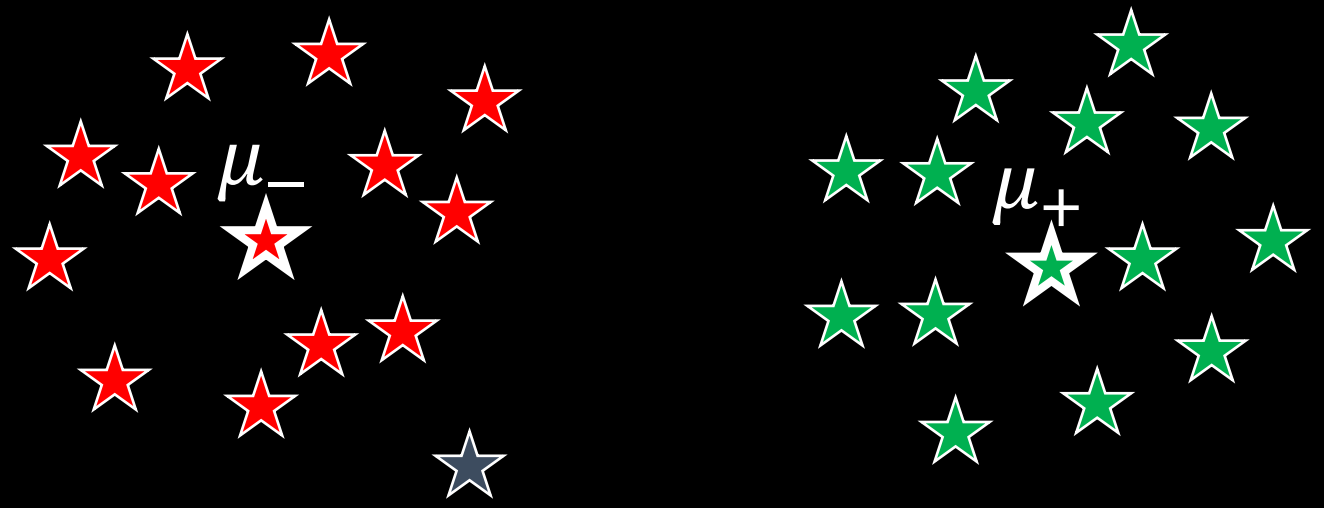
The choice of features and distance can change the prediction.

A new example inherits the label of its closest class prototype.

LwP: Mathematically

$$||\boldsymbol{\mu}_- - \mathbf{x}||^2 = ||\boldsymbol{\mu}_-||^2 + ||\mathbf{x}||^2 - 2\langle \boldsymbol{\mu}_-, \mathbf{x} \rangle$$

$$||\boldsymbol{\mu}_+ - \mathbf{x}||^2 = ||\boldsymbol{\mu}_+||^2 + ||\mathbf{x}||^2 - 2\langle \boldsymbol{\mu}_+, \mathbf{x} \rangle$$



Prediction Rule: Predict label as +1 if $f(\mathbf{x}) = ||\boldsymbol{\mu}_- - \mathbf{x}||^2 - ||\boldsymbol{\mu}_+ - \mathbf{x}||^2 > 0$ otherwise -1

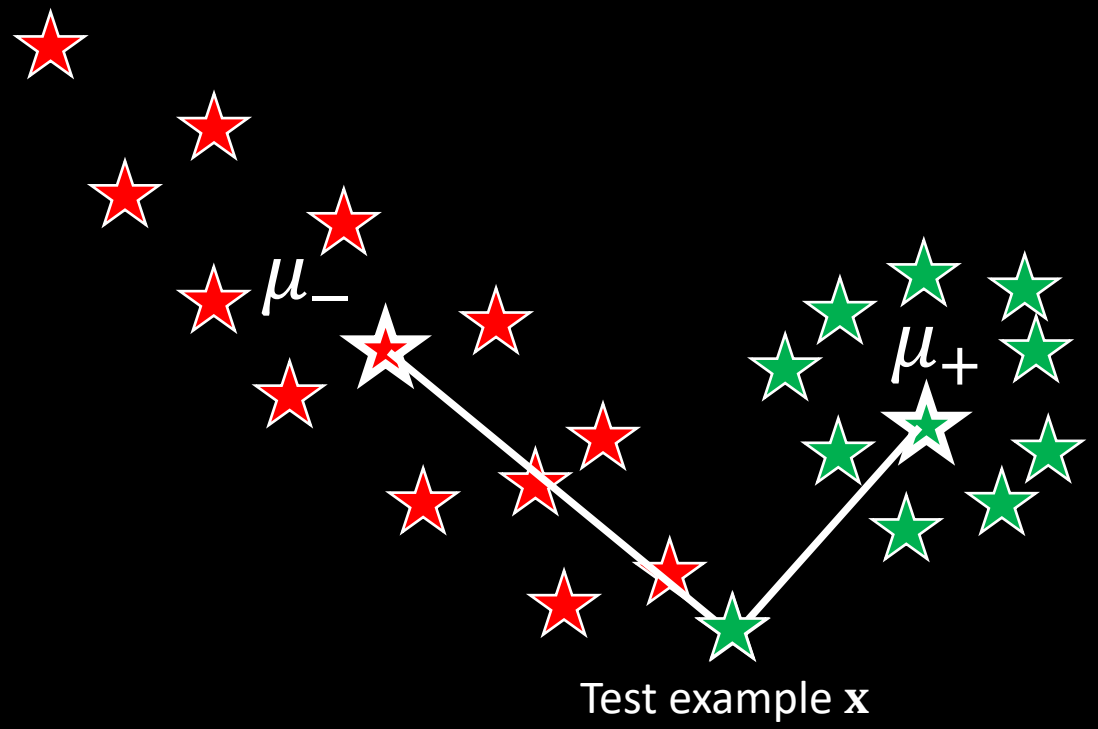
$$\begin{aligned} f(\mathbf{x}) &= ||\boldsymbol{\mu}_- - \mathbf{x}||^2 - ||\boldsymbol{\mu}_+ - \mathbf{x}||^2 \\ &= ||\boldsymbol{\mu}_-||^2 + ||\mathbf{x}||^2 - 2\langle \boldsymbol{\mu}_-, \mathbf{x} \rangle - ||\boldsymbol{\mu}_+||^2 - ||\mathbf{x}||^2 + 2\langle \boldsymbol{\mu}_+, \mathbf{x} \rangle \\ &= 2\langle \boldsymbol{\mu}_+ - \boldsymbol{\mu}_-, \mathbf{x} \rangle + ||\boldsymbol{\mu}_-||^2 - ||\boldsymbol{\mu}_+||^2 \\ &= \langle \mathbf{w}, \mathbf{x} \rangle + b \end{aligned}$$

Thus LwP with Euclidean distance is a linear model in the given features

A model of the form $f(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b$ is said to be linear in the input features $\mathbf{x}$



LwP Can Sometimes Fail (if used naïvely)



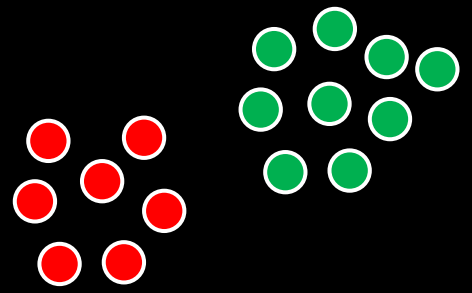
- But such failure cases can be prevented, e.g., using
  - Better distance function such as weighted Euclidean distance and standard Euclidean distance
  - Better similarity functions such as RBF kernels
  - Better feature extracted from the raw inputs (e.g., using a deep learning based feature extraction – more on this later)

LwP is simple but can be incredibly powerful if used correctly (can given even deep learning methods a run for their money 😊)

ML Model trained with different configurations

Here: Model = LwP, configuration = which features and which distance/similarity

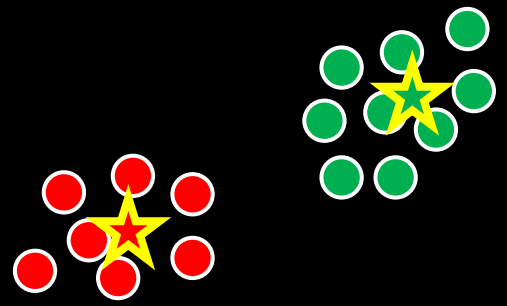
Training Data



LwP-1

A

Raw Pixels
+
Euclidean Distance

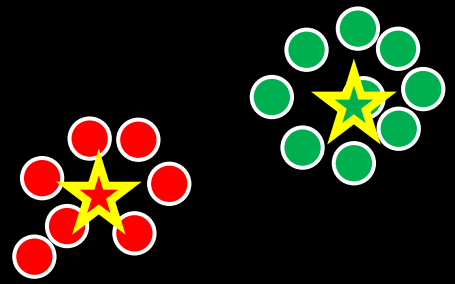


Training Accuracy = 92%

LwP-2

B

Raw Pixels
+
RBF Kernel Similarity

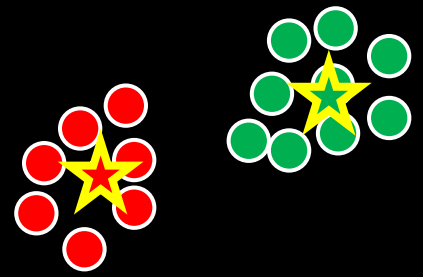


Training Accuracy = 97%

LwP-3

C

Learned Embeddings
+
Euclidean Distance



Training Accuracy = 95%

Best on training data ≠ best on future data

We need unseen examples to choose among configurations.

Training accuracy measures performance on examples the model has already seen.

So we can't rely on training accuracy to pick the best configuration.

We need something that can serve as a proxy of unseen future test data to choose among configurations.

Goal: Choose the configuration that is expected to perform best on unseen future test data.